Relatório 2 - Prática: Python: Criando um ReAct Agent do Zero (I)

Natalie Tolentino Serafim

Descrição da atividade

A atividade 2 envolveu assistir ao vídeo “Python: Create a ReAct Agent from Scratch” que explica a criação de um Agente de IA seguindo o modelo ReAct utilizando apenas a linguagem python e a biblioteca Groq para conexão com modelos de linguagem. Não foram utilizadas bibliotecas de apoio ao desenvolvimento de agentes dado o propósito de entender como o fluxo agêntico funciona.

O modelo ReAct estabelece um padrão de reflexão e ação para o agente. Nessa implementação foi utilizado a seguinte estrutura:

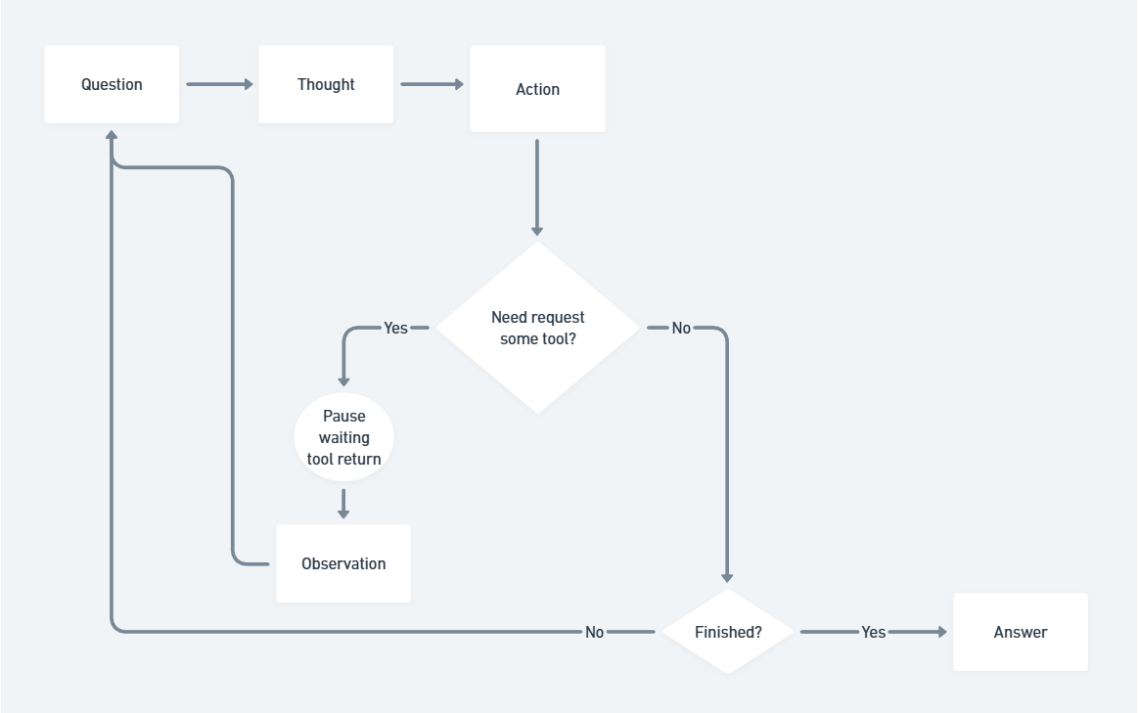

Figura 1 Fluxo do agente ReAct implementado. Autoria própria.

Esse fluxo estabelece ao agente um loop de Pensar, Agir, Esperar, Observar (Thought, Action, PAUSE, Observation). Neste fluxo, quando o agente recebe uma questão ele pensa a respeito, realiza alguma ação e avalia se precisa de alguma ferramenta externa, caso não exija e a tarefa esteja concluída ele responde ao usuário, mas caso seja necessário acionar uma ferramenta externa ele informa ao sistema e fica em pause, aguardando nova chamada com o retorno. Esse loop pode se repetir até que o modelo entenda que a resposta obtida responde ao que foi requisitado, ou até estourar o máximo de interações, sendo a definição padrão de 10 repetições.

Dessa forma a responsabilidade de avaliar os passos necessários para resolver o problema esteve com o LLM, e este por outro lado, atendia as instruções do fluxo estabelecido. Assim foi possível realizar tarefas mais complexas bem como acionar ferramentas externas.

Para o processamento de texto foi utilizado a plataforma Groq que permite interação com diversos modelos de linguagem, sendo o “llama-3.3-70b-versatile” escolhido para este caso.

A fim de instruir o agente a respeito do fluxo a ser seguido e quais ferramentas estariam disponíveis a ele, um system prompt foi definido explicando e exemplificando o que o agente deveria fazer.

Para permitir que o LLM fosse capaz de orquestrar o fluxo um conjunto de mensagens eram enviadas. A primeira mensagem indicando que possui a *role* de sistema e contendo o system prompt definido. A segunda mensagem referia-se a solicitação que o usuário fez. As respostas que o agente retornava eram adicionadas indicando a role 'assistent'. Assim cada chamada ao modelo levava consigo todo o histórico de interações, dando ao modelo o contexto do que foi solicitado, o que ele precisa fazer, o que já foi feito e assim indicando os próximos passos.

Dessa maneira, a aula em vídeo indicada ensina como construir um agente Python do zero.

Enquanto assistia ao vídeo, tomei notas do que compreendia, e ao finalizá-lo repliquei o código construído a fim de fixar o conhecimento. Em seguida, defini um possível tema para construir meu próprio agente e testar a fixação do estudo.

Para o meu agente defini como responsabilidade gerenciar uma lista de tarefas. O agente deveria reconhecer a entrada de texto como uma requisição para adicionar uma nova tarefa ou solicitação para listar as tarefas pendentes. Minha ideia aqui foi criar *tools* que escrevessem e lessem um arquivo .txt no computador, algo simples mas impossível ao modelo de linguagem sem a arquitetura de agente.

Dificuldades

Durante a escrita do meu agente presumi que seria de fácil modificação o código de exemplo da aula, afinal seria apenas escrever a função das novas tools e modificar o system prompt.

Por ser uma tarefa mais automática, deleguei ao ChatGPT reescrever as funções das tools, agora realizando a manipulação de arquivos, bem como já adaptar o system prompt dentro do cenário que estabeleci.

A questão é que o ChatGPT insistiu em modificar toda a estrutura do código, recomendar boas práticas e inserir features não solicitadas, como a extração de data da tarefa.

As sugestões tinham seu valor, por exemplo a recomendação de não utilizar a função *eval( )*, principalmente quando ela executará parte do texto de entrada do usuário.

Tentei fazer meu código com as sugestões ali apresentadas, mas não obtive sucesso.

O System prompt gerado pela IA até parecia estruturalmente como o do exemplo de aula, no entanto, algumas modificações alteraram a lógica, fazendo por exemplo o código entrar em loop sempre, devido ao não envio da condição de parada.

Debugar, encontrar esse erro foi um desafio, até então eu nunca tinha escrito um texto que pudesse causar bug em código. Perceber o valor da estrutura do sytem prompt e como mesmo a linguagem escrita pode mascarar erros lógicos foi uma surpresa para mim.

Depois de algumas tentativas fracassadas, optei por apagar e recomeçar meu código, usando agora por base somente o código de exemplo e fazendo somente as alterações que realmente necessitava. Ao final, reescrevi todo o código, passei por diversas etapas de debug, mas consegui enfim, implementar meu agente personalizado.

Conclusões

Com essa atividade foi possível compreender a arquitetura de um Agente de IA e perceber que mesmo sem outras libs é possível implementar uma arquitetura agêntica puramente com Python, lógica e consultas a um LLM.

Observo também meu estranhamento em construir uma lógica de execução por texto para o prompt do sistema. A dinâmica de modelos de linguagem me causa certa confusão dado meu costume pragmático de construir algoritmos determinísticos. Encontrar um erro lógico em uma explicação descritiva foi um desafio enriquecedor para adentrar ao novo mundo da IA.

Referencias

Alejandro AO. Python: Create a ReAct Agent from Scratch. YouTube, 17 de jul. de 2024. 57:26. Disponível em: https://www.youtube.com/watch?v=hKVhRA9kfeM. Acesso em: 17 de Fev. de 2026.